Diagnostic round trip — defects found end to end

System	HIS ERP — Medinova Medical Services Ltd. (UAT, `202.4.101.118:1115`)
Found by	`tests/E2E/diagnostic-round-trip.spec.ts` — one patient registered at the billing counter and taken to the delivery counter: investigation entry → sample collection → dispatch → acknowledgement → lab result entry → finalization (pathologist) → microbiology report → report delivery
Dates observed	8–10 November 2026
Accounts	`Admin02` (Administrator), `Sazia02` (Pathologist)

Why this report is written rather than generated

The defects in `sample-collection-defects.pdf` are states: a bad date in a box, a discount that should have been refused. `generate.js` can walk up to each one and photograph it.

These are not. Every finding below is something the system does **under load or over time** — a save that outruns a database timeout, a circuit that drops mid-form, a list that catches up three minutes late. A capture script run against a quiet server photographs a screen working perfectly. So the evidence here is what the automated journey actually recorded on the days it ran, quoted verbatim, with the invoice numbers to trace.

Summary

Two of these cost work. `E2E_B_01` makes a counter operator re-post a bill they have already filled in; `E2E_B_03` loses whatever is on screen without warning. Both are load-related and both got worse on the afternoon of 10 November, when the login POST alone took 53 seconds.

Two more are about the system **not saying what it has done**. `E2E_B_02` leaves a pathologist unable to tell from the screen whether a report was signed off. `E2E_B_05` points the counter at a tab the reports have not reached yet. Neither loses data; both invite someone to press the button a second time.

The remaining two are smaller, but `E2E_B_04` is the one to look at first if anyone has time for only one: a suggestion list that belongs to an earlier keystroke is indistinguishable from the right one, and the operator picking from it is choosing a **test to bill and perform**.

E2E_B_01 Posting a bill fails under load, and the operator must do it again

Severity	HIGH
Screen	OPD Investigation Entry (`/diagnostic/investigation`)
Observed	8 Nov 2026

Steps

1. Load a patient by UHID, add five tests (one of them a culture, which brings in its own tube).
2. Settle the bill in full in cash.
3. Press **POST** while the server is busy.

Expected — the bill is written, or it is not written and the reason is something the operator can act on.

Actual — the save is rolled back and the screen answers:

> Lab numbers could not be allocated, so nothing was saved. Please save again. (Execution Timeout Expired. The timeout period elapsed prior to completion of the operation or the server is not responding.)

What is right about it: nothing is half-written, and the message says so plainly and tells the operator what to do. The form is left intact, so a second POST succeeds.

Why it is still a defect: a bill is numbered by allocating a lab number per test, and that allocation is being done inside the request, serially, against a database command timeout it can outrun. Every test on the bill makes it more likely. At the counter this is a patient standing at the window while the operator posts the same bill twice.

Suggested look: allocate the lab numbers in one statement rather than per test, or raise the command timeout for this operation specifically.

E2E_B_02 A signed-off report looks exactly like one that was not signed off

Severity	MEDIUM
Screen	Lab Result Finalization (`/diagnosis/pathology-result-finalized`), as Pathologist
Observed	9 Nov 2026, twice, with different results

Steps

1. Open a pending report, correct a result, save it.
2. Note the buttons under the report.
3. Press **Finalize**.
4. Note the buttons again.

Expected — the screen says the report is signed off: Finalize goes, or goes dead, and **Undo** comes alive so the sign-off can be reversed.

Actual — it depends on the run, and neither outcome tells the operator anything.

First run:

```
buttons before finalizing: Save, Finalize, Undo (disabled), Clear Form
buttons after finalizing:  Save (disabled), Finalize (disabled), Undo (disabled), Clear Form
```

Second run, the app confirming the sign-off in as many words:

```
buttons before finalizing: Save, Finalize, Undo (disabled), Clear Form
snackbar: Successfully finalized.
buttons after finalizing:  Save, Finalize, Undo (disabled), Clear Form
```

Nothing changed at all in the second case. **Undo is disabled before and after, in every run** — it never becomes usable, so the screen offers no way to reverse a sign-off and no sign that one happened.

The only reliable confirmation is indirect: the invoice moves from the Pending tab to Done.

Why it matters: the pathologist has signed a clinical report and the screen still offers Finalize. The natural response is to press it again.

`E2E_B_03` The session drops mid-screen and the app silently returns to the module selector

Severity	HIGH
Screen	Any (seen on Lab Result Finalization and Microbiology Report)
Observed	9 Nov 2026

Steps — work on any screen for a few minutes while the server is loaded.

Expected — the circuit reconnects and the screen is where it was left, or the operator is told plainly that the connection dropped and what became of their work.

Actual — two different failures, depending on timing.

The circuit reconnects but the page has reloaded. The screen, its tabs and its fields are gone; the page reads **"Welcome back, Dr. Sazia Afreen — Select a module to get started."** Anything typed and not saved went with it. Nothing says so.

The circuit has not come back yet. Blazor's own overlay is up and swallows every click:

```
<div id="components-reconnect-modal"></div> intercepts pointer events
```

The page underneath looks completely normal — fields, buttons, data all drawn — so the operator sees a screen that simply stops responding.

Note for whoever picks this up: the application has its own reconnect overlay, `#reconnect-modal`, and Blazor's default one, `#components-reconnect-modal`, is also present and is the one that appeared here.

Worth checking that the custom overlay is actually wired to the circuit's disconnect events — if it were, the operator would get the app's own message rather than a page that ignores them.

`E2E_B_04` The test search shows the suggestions for an earlier keystroke

Severity	HIGH
Screen	OPD Investigation Entry — "Search test by name or code…"
Observed	9 Nov 2026

Steps

1. With the server busy, type `STOOL` into the test search box at a normal typing speed.
2. Read the suggestion list.

Expected — the list shows stool tests, or shows nothing while it is still fetching.

Actual — the list showed forty-odd **SERUM** tests: `SERUM FREE T4`, `SERUM FSH`, `SERUM GAMMA-GT`, `SERUM LIPID PROFILE (FASTING)`, `SERUM PROLACTIN` and so on. That is the answer to `S`, the first letter — the later keystrokes were still in flight, and the stale list sat there looking exactly like a finished one.

Why it matters: this is the control that chooses **which test the patient is billed for and the lab performs**. There is nothing on screen to distinguish a stale list from a current one — no spinner, no greying, no "searching…" — so an operator typing quickly and clicking the first plausible line can bill the wrong test. The automation only caught it because it was looking for one specific name and found none.

Suggested look: tag each request with the query it was for and discard answers that arrive for a query the box no longer holds; show the list as busy while a request is outstanding.

`E2E_B_05` A finalized report is announced at Report Delivery before it arrives there

Severity	MEDIUM
Screen	Microbiology Report → Report Delivery (`/diagnosis/report-delivery`)
Observed	9 Nov 2026

Steps

1. Finalize a culture report on the Microbiology screen. It answers: "*Report finalized. It is now in the Acknowledged tab of Report Delivery.*"
2. Open Report Delivery and search the invoice, within the minute.

Expected — the reports are where the message says they are.

Actual — Report Delivery showed nothing for that invoice under any of its three stages:

```
Not Acknowledged 0     Acknowledged 0     Delivery 0
```

Checked again about ten minutes later, with nothing else touched, the invoice was sitting under **Not Acknowledged** with all five of its reports. Two independent screens agreed it was missing in the meantime — Lab Report Published Group Wise did not have it either.

Why it matters: the message sends the counter to a tab the report has not reached. The patient is at the window; the clerk is looking at an empty list and has no way to tell "not ready yet" from "something went wrong".

`E2E_B_06` An invoice stays listed under a stage it has no reports left in

Severity	LOW
Screen	Report Delivery (`/diagnosis/report-delivery`)
Observed	9 Nov 2026

Steps

1. Acknowledge the only report an invoice has under **Not Acknowledged**.
2. Come back to **Not Acknowledged** and search that invoice.

Expected — the invoice is no longer listed there; it has nothing at that stage.

Actual — the invoice is still in the left-hand list and the row still selects. Clicking it loads an empty report grid with a blank patient bar beside it. Nothing explains the empty screen.

This is the same stale-count behaviour the existing suite records as `RD_B_03`, seen from the other side: there the counts beside the radio buttons kept a dropped invoice's figures, here the invoice itself stays in the list.

`E2E_B_07` "Print Label" collects nothing, and says nothing

Severity	HIGH
Screen	Sample Collection (`/diagnostic/sample-collection`)
Observed	10 Nov 2026

Steps

1. Open a freshly billed invoice under **New Patient**. Every test is ticked on opening.
2. Press **Print Label** — which is what records the samples as collected.
3. Re-open the invoice under **All Patient** and read the SAMPLE STATUS column.

Expected — each ticked sample reads "Label Printed" or further on, and the invoice moves to **Collected Patient**.

Actual — all four samples still read **Not Collected**:

```
21873834 | STOOL FOR R/E & M/E                     | Not Collected
21873833 | BLOOD FOR TC, DC, HB% & ESR - (MANUAL)  | Not Collected
21873835 | SERUM LIPID PROFILE (FASTING)           | Not Collected
21873836 | STOOL FOR CULTURE & SENSITIVITY         | Not Collected
```

The press went through, the grid redrew, and nothing had happened. Invoice 2611100200003.

And the screen says nothing either way. No snackbar is raised when the collection works and none when it does not — confirmed on the retry, which collected all four samples in silence. So there is no signal at all for an operator to go by: a successful collection and a failed one look and sound identical, and the only way to know which happened is to re-open the invoice and read the status column.

Why it matters: this is the step that tells the lab a sample exists. An operator who presses the button, sees the grid redraw and walks away has left four samples uncollected with nothing to suggest anything went wrong. Everything downstream — dispatch, acknowledgement, results — waits on a status that was never set.

The same button worked on half a dozen other runs with the same invoice shape, so the failure is intermittent rather than outright. The collection spec in this suite has carried a note about it since it was written; this is the first time it took a whole invoice.

Two things are worth separating here: the intermittent failure to record the collection, and the complete absence of feedback that makes it invisible. The second is the cheaper fix and would make the first survivable.

Note: POST on the billing screen reports `Could not find printer.` on this server, so no physical printing happens at all. If label printing and sample collection share a code path, it is worth checking whether a printer failure can abandon the collection without reporting it.

Also checked, and correct

- The whole round trip completes. One patient, one invoice, eight screens, five reports raised, sampled, resulted, signed off and handed over, with the status on each screen matching the work done on the one before it.
- **Nothing is half-written when a save fails.** The allocation timeout in `E2E_B_01` rolls the bill back completely; a second POST produces one bill, not two.
- **Double-pressing Save and Finalize does not file a report twice.** Both are pressed as a double-click by the automation on the microbiology report; the invoice appears exactly once under Done.
- **Sample status is carried correctly across the four sample screens.** Only collected samples are offered for dispatch, only dispatched ones reach the lab's acknowledgement list, and each screen's filters move the invoice between them as expected.
- **The X-ray is handled correctly throughout.** It takes no sample, so it is absent from collection, dispatch and acknowledgement, and present at result entry and at the counter.

- **Reports reach the counter at the right stage.** Lab and culture reports go straight to **Acknowledged** on sign-off; the radiology one waits under **Not Acknowledged** for someone to receive it. Delivery stamps every report with a time, and printing a delivered report changes nothing about it.
- **SMS is sent on acknowledgement** — "*Report ready SMS sent to 017…*" — against the mobile number on the bill.
- **The account switch is clean.** Signing the pathologist in ends the administrator's session on the server rather than leaving two live, and the menu each account sees matches its permissions.

How often these happen

Every finding here is intermittent. None of them is a screen that is simply broken — each is a screen that works until the server is under enough load, and then does not.

The clearest measure of that is 11 November: the whole journey ran green on the first attempt, all eight stages, ten and a half minutes, registering **MASUD RANA-1** at the counter and handing five finished reports over at the end of it. Not one of the seven findings appeared. The afternoon of 10 November, when the sign-in alone took 53 seconds, produced three of them inside an hour.

That is worth saying plainly for two reasons. A developer who opens any of these on a quiet morning will not be able to make it happen, and may close it as unreproducible. And a UAT sign-off taken on a quiet morning says nothing about what the counter will see at eleven o'clock on a busy one — which is exactly when `E2E_B_01` makes an operator post the same bill twice and `E2E_B_07` loses four samples in silence.

Reproducing

```
npm run test:e2e
```

Runs the whole journey. It registers its own patient on the billing screen — the demographics and the mobile number come from `test-data/e2e.json`, and the name carries a number that goes up on every run — then bills them and follows the work to the delivery counter.

It writes real data: a patient is registered, a bill is posted, samples are collected and dispatched, clinical results are filed and signed off, reports are handed over. Each run raises its own invoice, so repeating it is safe.

To make the load-related findings likely, run it against a busy server, or run several at once.